

MovieLens Project Report

Fabian Hiernaux

2026-05-03

Contents

1	Introduction	2
1.1	Project Objective	2
1.2	Environnement et contraintes techniques	3
2	Methods and Analysis	3
2.1	Data Preparation	3
2.2	Data Inspection & Computational Optimization	4
2.2.1	Dataset Dimensions and Basic Statistics	5
2.2.2	Visualizing the Rating Distribution	6
2.2.3	Overcoming Memory Constraints: Genre Analysis	7
2.2.4	Specific Findings	8
2.2.5	Psychological Bias: Whole vs. Half Stars	9
2.3	Feature Engineering	10
3	Advanced Exploratory Data Analysis: Identifying Systemic Biases	10
3.1	The Movie Effect: Quality Variation	10
3.2	The User Effect: Harsh vs. Generous Graders	11
3.3	Temporal Trends: The Time and Age Effects	12
3.3.1	Weekly Rating Trends	12
3.3.2	Movie Age and Release Year	13
3.4	Additional Explorations and Feature Selection	14
3.4.1	User Activity vs. Rating Quality	14
3.4.2	Movie Popularity vs. Rating Average	15
3.4.3	Weekday Rating Patterns	16
3.4.4	Title Length and Metadata	17
3.4.5	Multi-Genre Complexity	18
3.4.6	Summary of Excluded Variables	19

4	Modeling Results and Evolution	19
4.1	Evaluation Metric: RMSE	20
4.2	From Baseline to Complexity	20
4.2.1	Data Partitioning for Model Development	20
4.2.2	Model 1: The Global Average (μ).	21
4.2.3	Model 2: Adding Movie Effect (b_i).	21
4.2.4	Model 3: Adding User Effect (b_u).	22
4.2.5	Model 4: Adding Genre Effect (b_g).	23
4.2.6	Model 5: Adding Release Year Effect (b_y).	25
4.2.7	Model 6: Adding Temporal Effect (b_t).	26
4.2.8	Model 7: The Final Model with Regularization and Backfitting	27
5	Final Model Validation (Hold-out Test)	31
5.1	Final Feature Engineering	31
5.2	The Final Verdict	32
6	Conclusion	34
6.1	Summary of the Work	34
6.2	Final Results and Achievements	34
6.3	Limitations and Future Work	34
7	OLD STUFF - on garde pour vérifier ultérieurement si intérêt	35
7.1	Exploratory Data Analysis (EDA)	35
7.1.1	The User Effect	35
7.1.2	The Movie Release Year Effect	35

1 Introduction

The goal of this project is to develop a movie recommendation system using the MovieLens 10M dataset. In a world of overflowing digital content, recommendation algorithms are essential tools that help users discover items they are likely to enjoy based on their previous behavior and the behavior of others. The main objective is to develop an algorithm capable of predicting user ratings with a high degree of accuracy, as measured by the root mean square error (RMSE).

1.1 Project Objective

The primary mission is to build a machine learning model capable of predicting movie ratings with high precision. Accuracy is measured by the Root Mean Square Error (RMSE). Our target is to achieve a final RMSE lower than 0.86490.

1.2 Environnement et contraintes techniques

A significant aspect of this work is the technical constraint under which it was performed. The analysis was conducted on a Virtual Machine (based on virtualbox) limited to 6 GB of RAM and 4CPUs.

This constraint influenced the modeling strategy:

- **Efficiency:** We prioritized a regularized linear model optimised by backfitting over more memory-intensive techniques. This approach offers an excellent balance between predictive performance and resource consumption, whereas more resource-intensive methods (such as complex matrix factorisation) might have overloaded the system.
- **Optimization:** Explicit memory management (using `gc()` to clean up memory) was integrated into the workflow to ensure the system remained stable despite the large dataset size (10 million entries).

2 Methods and Analysis

2.1 Data Preparation

As per the instructions provided by the HarvardX course, the initial code to download the MovieLens 10M dataset and split it into two sets was used:

1. **edx set:** Used for data exploration, model training, and parameter tuning.
2. **final_holdout_test set:** Kept strictly separate and used only for the final assessment of the model's accuracy.

```
# Note: this process could take a couple of minutes

if(!require(tidyverse)) install.packages("tidyverse", repos =
  ↪ "http://cran.us.r-project.org")
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-project.org")

library(tidyverse)
library(caret)

# MovieLens 10M dataset:
# https://grouplens.org/datasets/movielens/10m/
# http://files.grouplens.org/datasets/movielens/ml-10m.zip

options(timeout = 120)

dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file))
  unzip(dl, ratings_file)

movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file))
  unzip(dl, movies_file)
```

```

ratings <- as.data.frame(str_split(read_lines(ratings_file), fixed("::"), simplify =
  ↪ TRUE),
                        stringsAsFactors = FALSE)
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

movies <- as.data.frame(str_split(read_lines(movies_file), fixed("::"), simplify = TRUE),
                        stringsAsFactors = FALSE)
colnames(movies) <- c("movieId", "title", "genres")
movies <- movies %>%
  mutate(movieId = as.integer(movieId))

movielens <- left_join(ratings, movies, by = "movieId")

# Final hold-out test set will be 10% of MovieLens data
set.seed(1, sample.kind="Rounding") # if using R 3.6 or later
# set.seed(1) # if using R 3.5 or earlier
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

# Make sure userId and movieId in final hold-out test set are also in edx set
final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

# Add rows removed from final hold-out test set back into edx set
removed <- anti_join(temp, final_holdout_test)
edx <- rbind(edx, removed)

rm(dl, ratings, movies, test_index, temp, movielens, removed)

```

2.2 Data Inspection & Computational Optimization

Before building the model, we performed a rigorous inspection of the `edx` dataset to validate its structure and content. This initial exploration validates the data structure and answers the project's fundamental questions through direct code execution.

In summary, the key questions and their answers are as follows:

- **Dataset Dimensions:** The dataset contains **9,000,055** rows and **6** columns.
- **Rating Values:**
 - No ratings of **0** were given, which is consistent with the MovieLens scale (0.5 to 5).
 - There are **2,121,240** ratings of **3**.
- **Unique Entries:** There are **10,677** unique movies and **69,878** unique users.
- **Genre Distribution:** Drama is the most rated genre (**3,910,127**), followed by Comedy (**3,540,930**), Thriller (**2,325,899**), and Romance (**1,712,100**).
- **Most Popular Movie:** *Pulp Fiction (1994)* holds the record for the greatest number of ratings.

- **Rating Frequency:** The five most common ratings are, in order: **4, 3, 5, 3.5, and 2.**
- **Rating Type Bias:** As observed, half-star ratings are significantly less common than whole-star ratings, suggesting a psychological preference for integer scores.

2.2.1 Dataset Dimensions and Basic Statistics

We first verified the volume of data and the range of ratings.

```
# 1. Structure and dimensions of edx
res_dim <- dim(edx)
message(paste("The edx dataset has", res_dim[1], "rows and", res_dim[2], "columns."))
```

```
# 2. Preview of the data
head(edx)
```

```
##   userId movieId rating timestamp                title
## 1      1      122      5 838985046          Boomerang (1992)
## 2      1      185      5 838983525            Net, The (1995)
## 4      1      292      5 838983421            Outbreak (1995)
## 5      1      316      5 838983392            Stargate (1994)
## 6      1      329      5 838983392 Star Trek: Generations (1994)
## 7      1      355      5 838984474    Flintstones, The (1994)
##                                     genres
## 1                                Comedy|Romance
## 2                   Action|Crime|Thriller
## 4  Action|Drama|Sci-Fi|Thriller
## 5                   Action|Adventure|Sci-Fi
## 6 Action|Adventure|Drama|Sci-Fi
## 7          Children|Comedy|Fantasy
```

```
# 3. Summary of key statistics
summary(edx)
```

```
##      userId      movieId      rating      timestamp
## Min.   :      1  Min.   :      1  Min.   :0.500  Min.   :7.897e+08
## 1st Qu.:18124  1st Qu.:   648  1st Qu.:3.000  1st Qu.:9.468e+08
## Median :35738  Median :  1834  Median :4.000  Median :1.035e+09
## Mean   :35870  Mean   :   4122  Mean   :3.512  Mean   :1.033e+09
## 3rd Qu.:53607  3rd Qu.:  3626  3rd Qu.:4.000  3rd Qu.:1.127e+09
## Max.   :71567  Max.   :65133  Max.   :5.000  Max.   :1.231e+09
##      title      genres
## Length:9000055  Length:9000055
## Class :character  Class :character
## Mode  :character  Mode  :character
##
##
##
```

```
# 4. Counting unique users and movies, numbers of zeros and threes
n_unique_users <- n_distinct(edx$userId)
n_unique_movies <- n_distinct(edx$movieId)
```

```

message(paste("There are", n_unique_users, "unique users and", n_unique_movies, "unique
↳ movies in edx.))

# Verification of specific ratings. Using efficient vectorized sums for specific counts
count_0 <- sum(edx$rating == 0)
count_3 <- sum(edx$rating == 3)
# Insight: In the MovieLens 10M dataset, ratings are on a 0.5 to 5.0 scale.
# A count of 0 for the "0" rating confirms the expected range.
message(paste("Number of 0 ratings:", count_0))
message(paste("Number of 3 ratings:", count_3))

```

The inspection of the `edx` dataset reveals a massive volume of data, consisting of **9,000,055** rows and **6** columns.

Among these records, we identify:

- **69,878** unique users.
- **10,677** unique movies.

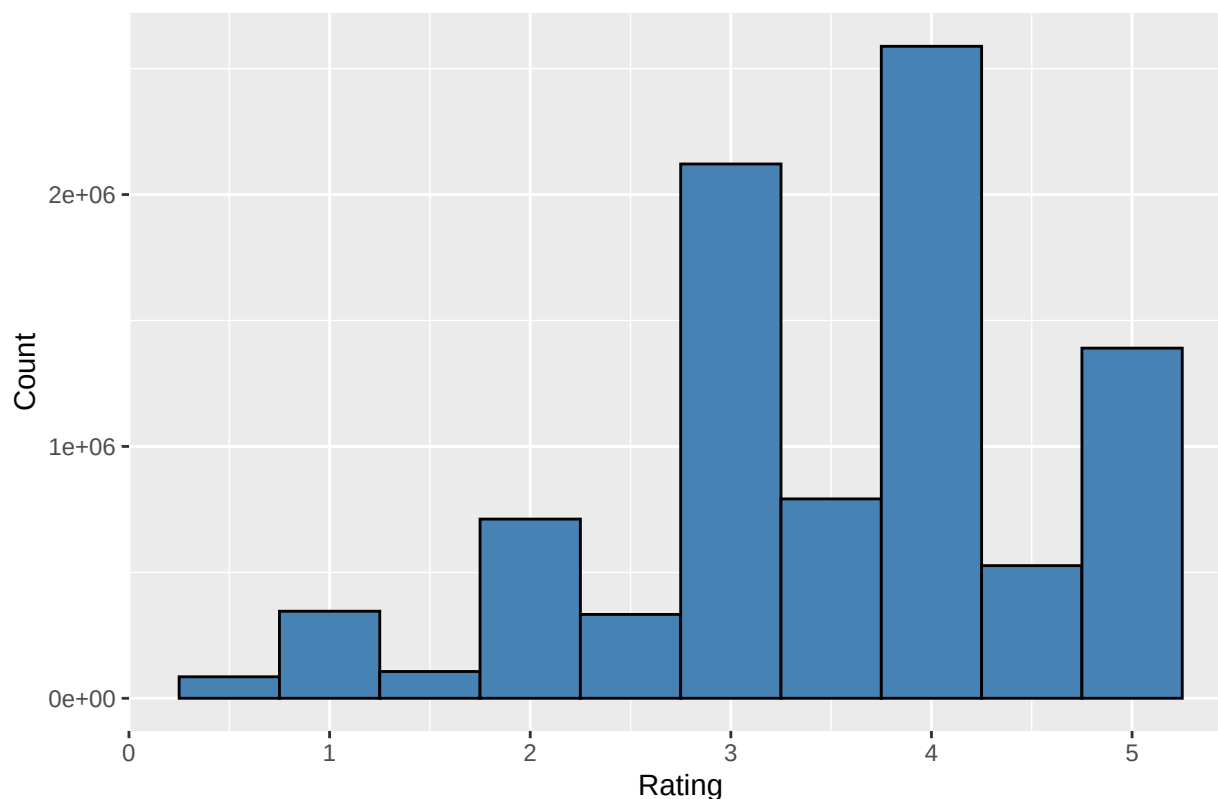
Regarding the rating values, we confirmed the integrity of the scale: there are **0** ratings of zero (as expected for a 0.5 to 5.0 scale), while the score of 3 appears **2,121,240** times.

2.2.2 Visualizing the Rating Distribution

The following visualization helps determine if users tend to be generous or strict, and highlights the preference for specific types of scores.

How do people rate movies?

General Distribution of Ratings



The histogram reveals a “positivity bias,” with most ratings above 3. Furthermore, we can clearly see that whole-star ratings are more frequent than half-star ratings.

2.2.3 Overcoming Memory Constraints: Genre Analysis

One of the main technical challenges of this project was the 6 GB RAM limitation of the Virtual Machine. A standard Tidyverse approach to count genres (using `separate_rows()` on 9 million rows) causes system instability due to memory exhaustion.

To identify the top genres, we evaluated three different algorithms to balance accuracy and performance:

- Method 1 (Sampling): Taking a 1M row sample for quick estimation.
- Method 2 (String Detection): Using `str_detect` for targeted counting.
- Method 3 (Pre-aggregation): The most efficient approach. By grouping the data by unique genre combinations before expanding them, we reduced the workload from 9 million rows to only ~800 unique strings.

We chose Method 3 for its mathematical identity to the standard approach while maintaining a minimal memory footprint.

```
# Optimal Engineering approach (Method 3)
top_genres_m3 <- edx %>%
  group_by(genres) %>%
  summarize(n = n()) %>%
```

```

separate_rows(genres, sep = "\\|") %>%
group_by(genres) %>%
summarize(count = sum(n)) %>%
arrange(desc(count))

head(top_genres_m3, 10)

```

```

## # A tibble: 10 x 2
##   genres      count
##   <chr>      <int>
## 1 Drama    3910127
## 2 Comedy   3540930
## 3 Action   2560545
## 4 Thriller 2325899
## 5 Adventure 1908892
## 6 Romance  1712100
## 7 Sci-Fi   1341183
## 8 Crime    1327715
## 9 Fantasy   925637
## 10 Children 737994

```

The analysis confirms that Drama is the most rated genre, followed by Comedy, Action and Thriller.

2.2.4 Specific Findings

Finally, we identified the most popular items and the frequency of specific scores to confirm our modeling assumptions.

```

# Most rated movie
edx %>%
  group_by(movieId, title) %>%
  summarize(count = n()) %>%
  arrange(desc(count)) %>%
  head(1)

```

```

## # A tibble: 1 x 3
## # Groups:   movieId [1]
##   movieId title                count
##   <int> <chr>                  <int>
## 1     296 Pulp Fiction (1994) 31362

```

```

# Top 5 most given ratings
edx %>%
  group_by(rating) %>%
  summarize(count = n()) %>%
  arrange(desc(count)) %>%
  head(5)

```

```

## # A tibble: 5 x 2
##   rating count

```

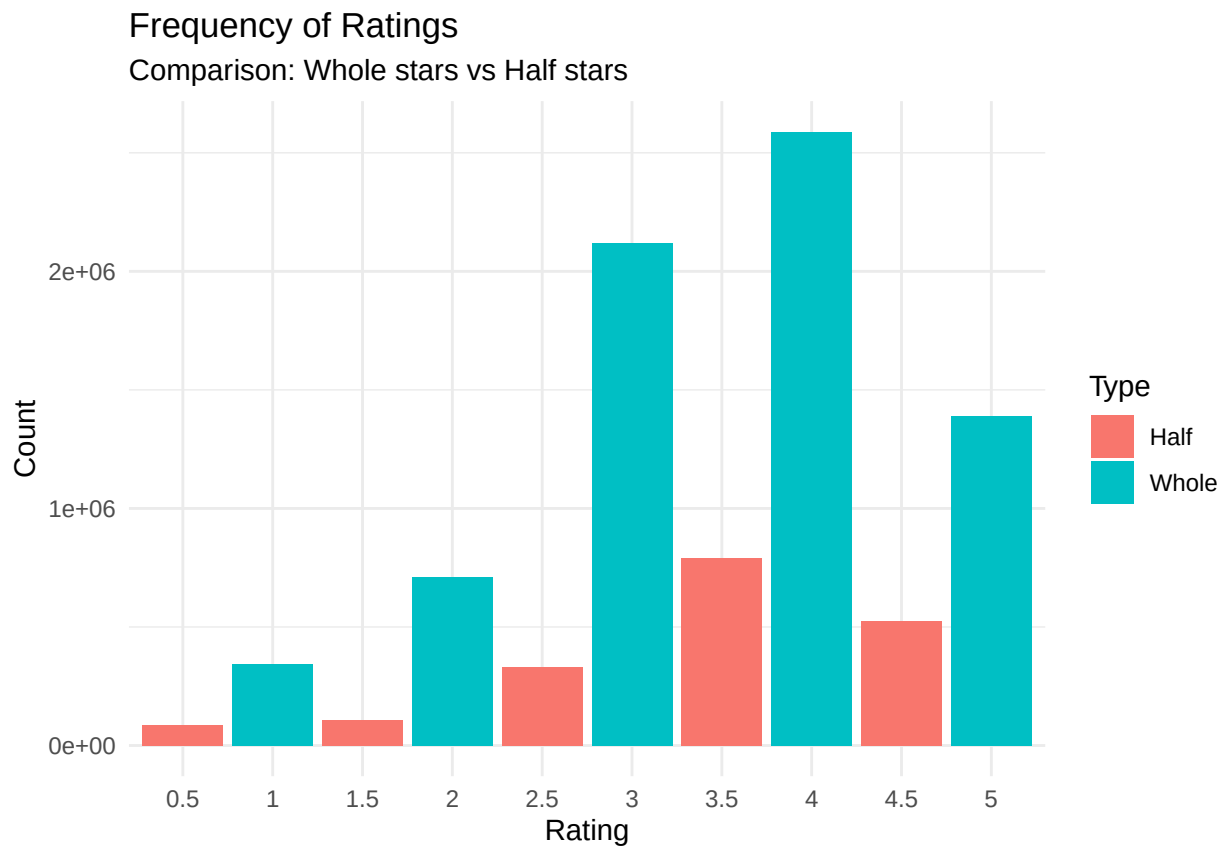


```
##      <dbl>      <int>
## 1      4    2588430
## 2      3    2121240
## 3      5    1390114
## 4     3.5    791624
## 5      2    711422
```

2.2.5 Psychological Bias: Whole vs. Half Stars

The visualization below reinforces the observation that users have a psychological preference for integer ratings.

```
edx %>%
  group_by(rating) %>%
  summarize(count = n()) %>%
  mutate(half_star = ifelse(rating %% 1 == 0, "Whole", "Half")) %>%
  ggplot(aes(x = factor(rating), y = count, fill = half_star)) +
  geom_bar(stat = "identity") +
  labs(title = "Frequency of Ratings",
       subtitle = "Comparison: Whole stars vs Half stars",
       x = "Rating", y = "Count", fill = "Type") +
  theme_minimal()
```



2.3 Feature Engineering

To move beyond a simple average, we extracted two key temporal variables from the raw data:

- **Release Year:** Extracted from the movie title to see if “classics” are rated differently than modern blockbusters.
- **Review Week (Date):** Extracted from the timestamp to capture trends in rating behavior over time.

Before proceeding with the exploratory analysis, we transform the raw data to extract specific features. This step is essential because our final modeling strategy (detailed in Section 4) will move beyond simple averages to incorporate temporal and movie-age effects. By extracting the release year from the titles and converting the timestamps into weekly dates at this stage, we can visualize these trends before formally integrating them into our regularized model.

```
# Extracting release year and converting timestamp to weekly dates  
# This ensures that these columns are available for both EDA and modeling  
  
edx <- edx %>%  
  mutate(release_year = as.numeric(str_extract(str_extract(title, "\\(\\d{4}\\)$"),  
    ↪ "\\d{4}")),  
         date = round_date(as_datetime(timestamp), unit = "week"))  
  
final_holdout_test <- final_holdout_test %>%  
  mutate(release_year = as.numeric(str_extract(str_extract(title, "\\(\\d{4}\\)$"),  
    ↪ "\\d{4}")),  
         date = round_date(as_datetime(timestamp), unit = "week"))
```

3 Advanced Exploratory Data Analysis: Identifying Systemic Biases

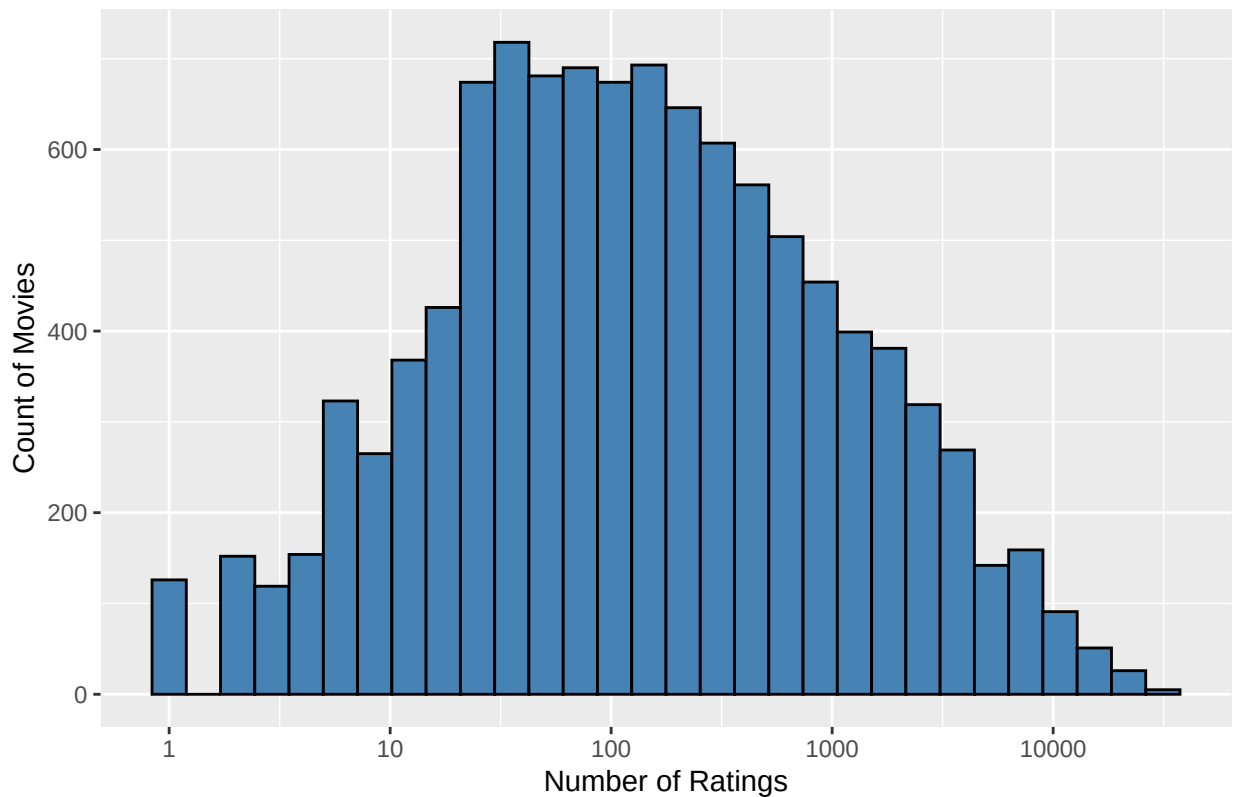
The initial inspection confirmed the basic structure of the data, but it also hinted at deeper patterns. To build an accurate recommendation system, we must move beyond simple averages. In this section, we analyze the specific “biases” (systemic variations) that influence how a movie is rated.

These findings directly inform our modeling strategy: for each pattern observed here, a corresponding mathematical component will be integrated into our final algorithm.

3.1 The Movie Effect: Quality Variation

Not all movies are created equal. Some are universally acclaimed, while others are poorly received.

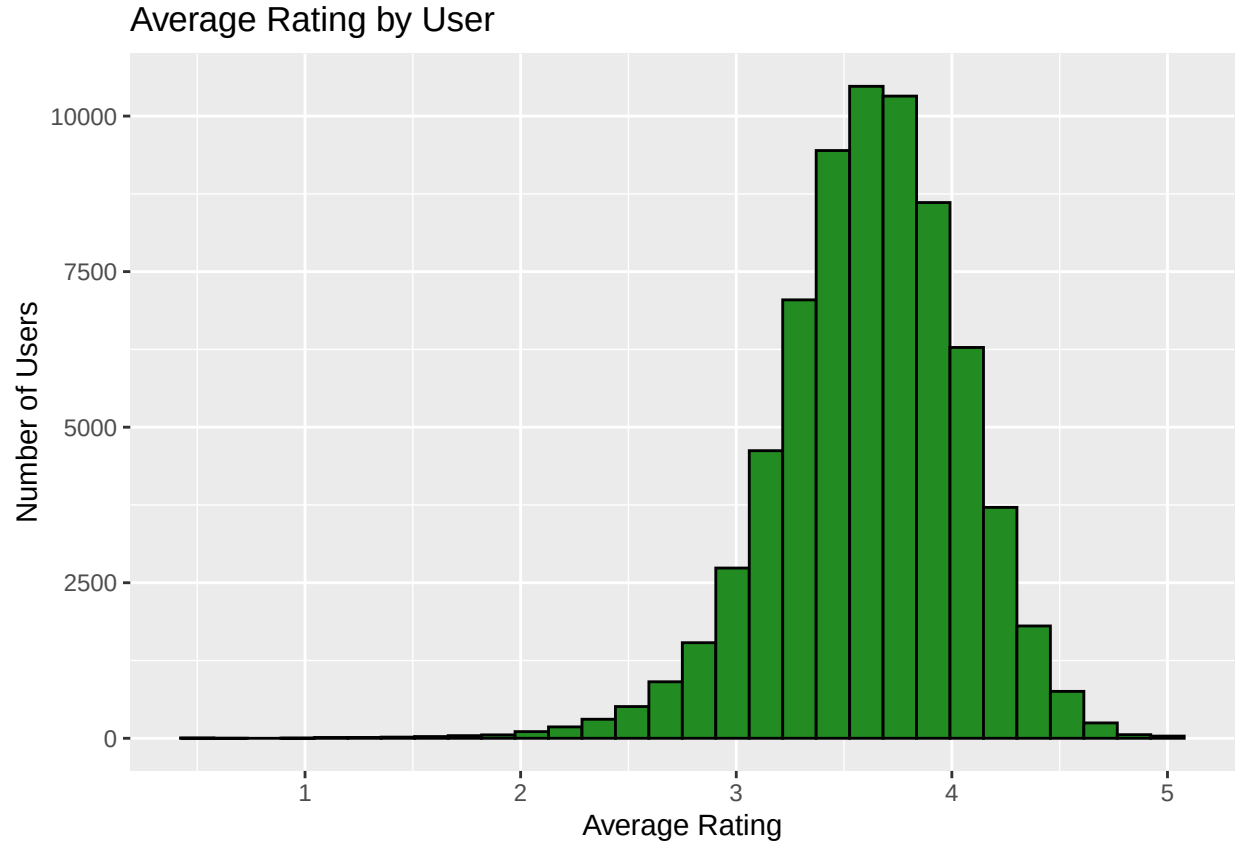
Distribution of Ratings per Movie



The histogram above shows that while some movies have thousands of ratings, many others have very few. This justifies the inclusion of a **Movie Bias (μ_i)** and highlights the need for **Regularization** to prevent movies with very few ratings from skewing the results.

3.2 The User Effect: Harsh vs. Generous Graders

Users have different rating scales. A “3” from a very critical user might be equivalent to a “5” from a more lenient one.

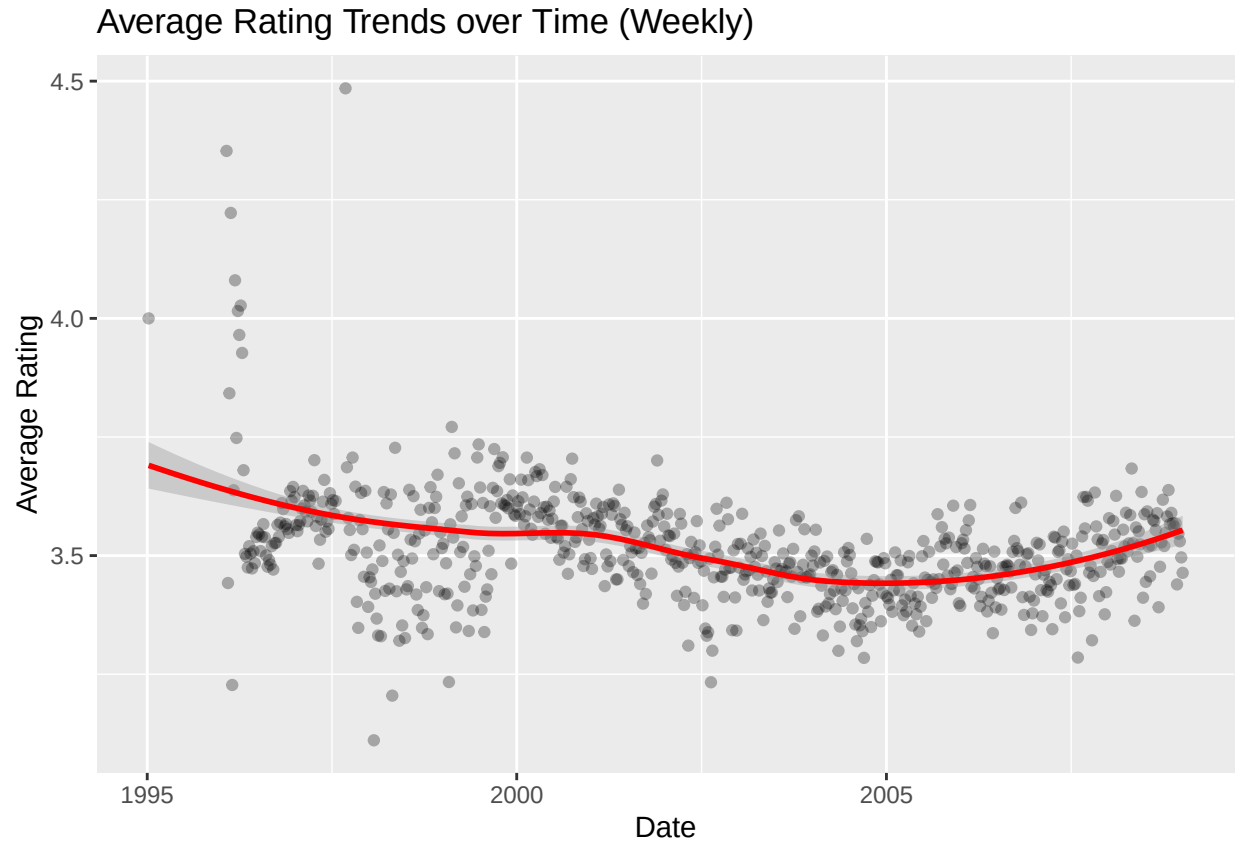


The distribution of average ratings per user confirms a significant **User Effect** (b_u). Some users consistently rate below the mean, while others are much more generous.

3.3 Temporal Trends: The Time and Age Effects

3.3.1 Weekly Rating Trends

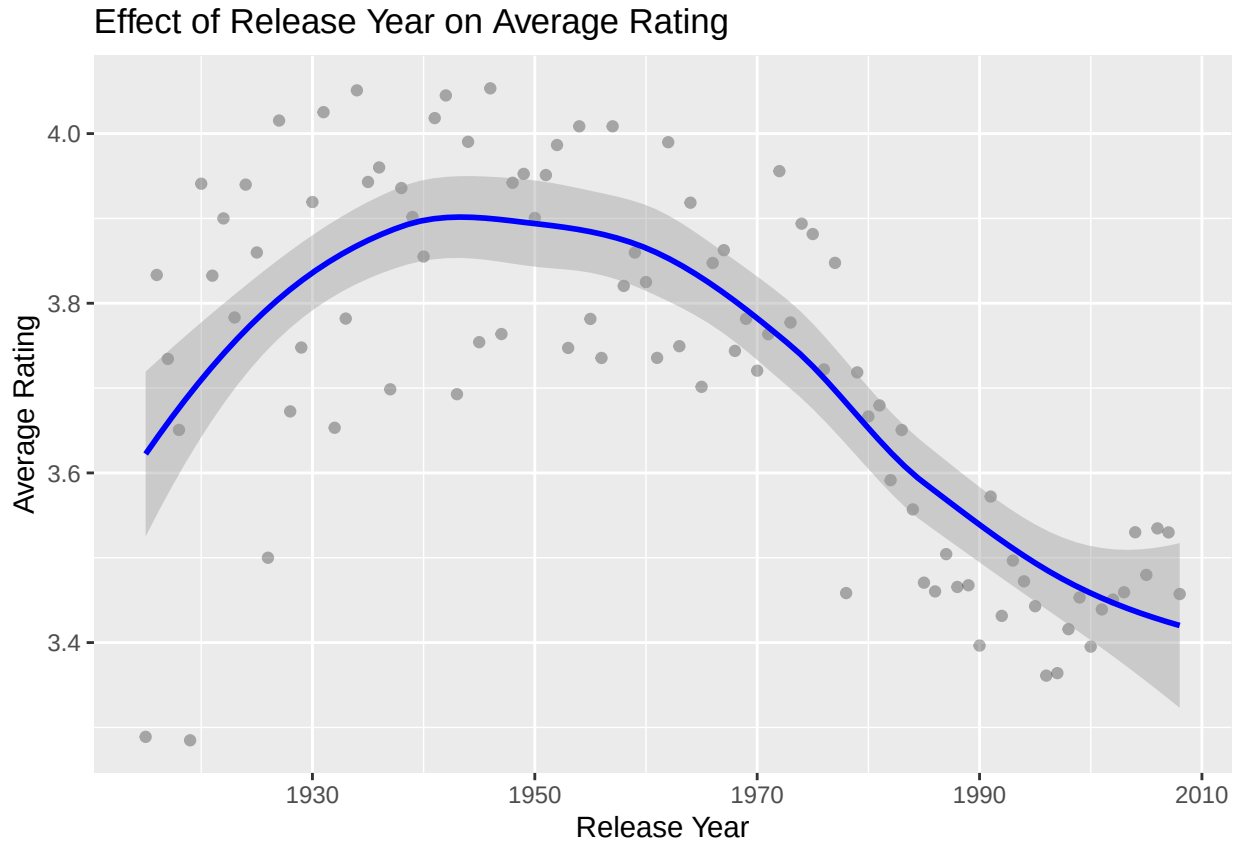
Does the general “mood” of the community change over time? We analyzed the average rating on a weekly basis.



There is a visible, albeit subtle, fluctuation in average ratings over the years. This supports the inclusion of a **Temporal Bias** (b_t) in our advanced models.

3.3.2 Movie Age and Release Year

We also investigated if older movies are rated differently than newer ones.



Interestingly, older movies (released before 1980) tend to have higher average ratings, possibly due to a “classic” status or survival bias (only good old movies are still being watched and rated). This confirms the **Release Year Effect** (b_y).

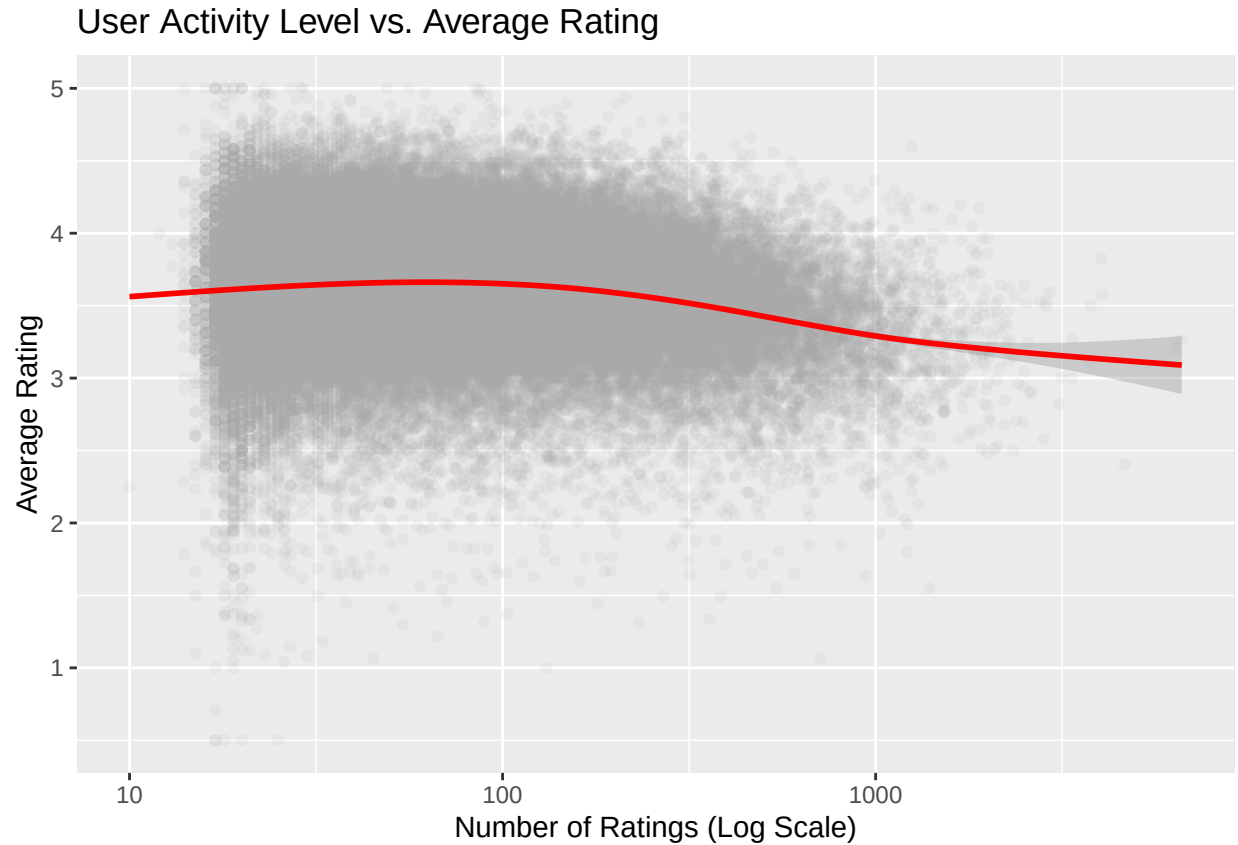
3.4 Additional Explorations and Feature Selection

To ensure the robustness of our model, several other variables were investigated. While these analyses provided valuable insights into user behavior, they were not included in the final model either because they provided redundant information or because their impact on the RMSE was negligible compared to the increased computational complexity.

3.4.1 User Activity vs. Rating Quality

We examined whether prolific users (those who rate many movies) tend to be more critical or more generous.

```
edx %>%
  group_by(userId) %>%
  summarize(n = n(), avg = mean(rating)) %>%
  ggplot(aes(n, avg)) +
  geom_point(alpha = 0.1, color = "darkgrey") +
  scale_x_log10() +
  geom_smooth(color = "red") +
  labs(title = "User Activity Level vs. Average Rating",
       x = "Number of Ratings (Log Scale)", y = "Average Rating")
```

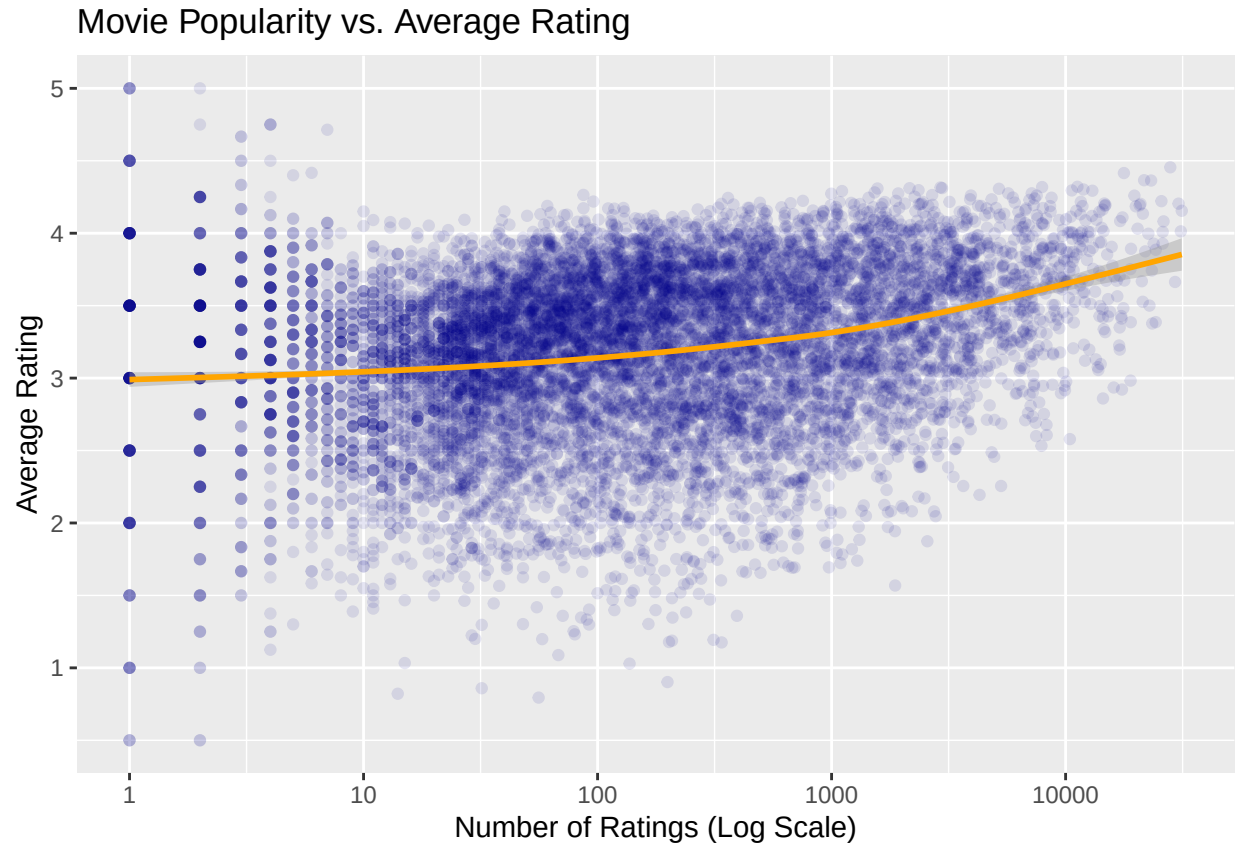


Observation: There is a slight trend where very active users tend to rate closer to the overall mean. However, this effect is largely captured by the User Bias (bu) already present in our model.

3.4.2 Movie Popularity vs. Rating Average

We tested if “Blockbusters” (highly rated movies) generally receive higher scores.

```
edx %>%
  group_by(movieId) %>%
  summarize(n = n(), avg = mean(rating)) %>%
  ggplot(aes(n, avg)) +
  geom_point(alpha = 0.1, color = "darkblue") +
  scale_x_log10() +
  geom_smooth(color = "orange") +
  labs(title = "Movie Popularity vs. Average Rating",
       x = "Number of Ratings (Log Scale)", y = "Average Rating")
```

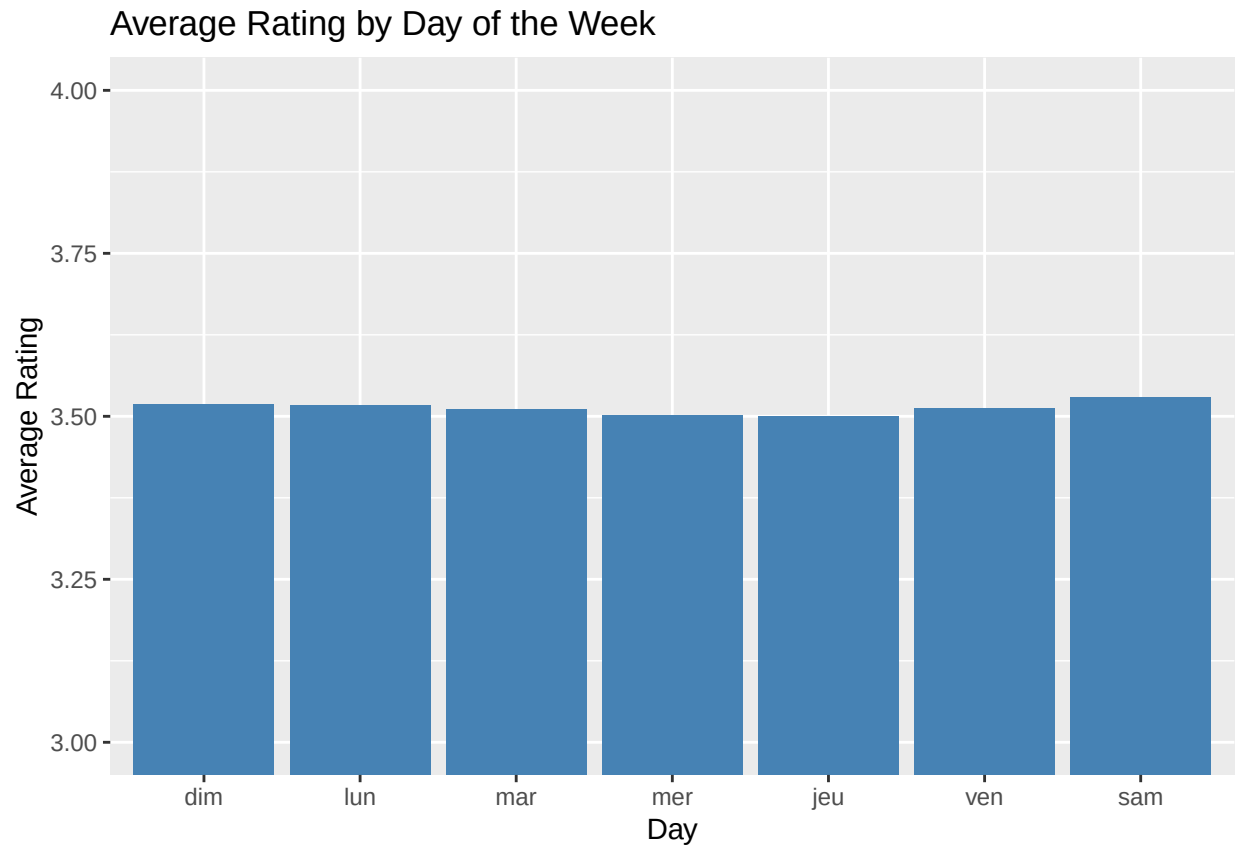


Observation: Popular movies do tend to have higher averages, but this correlation is implicitly handled by the Movie Bias (bi). Adding a specific “popularity” variable would introduce multicollinearity.

3.4.3 Weekday Rating Patterns

We analyzed if the day of the week (Monday to Sunday) influenced the ratings.

```
edx %>%
  mutate(dow = wday(as_datetime(timestamp), label = TRUE)) %>%
  group_by(dow) %>%
  summarize(avg = mean(rating), n = n()) %>%
  ggplot(aes(dow, avg)) +
  geom_col(fill = "steelblue") +
  coord_cartesian(ylim = c(3, 4)) +
  labs(title = "Average Rating by Day of the Week", x = "Day", y = "Average Rating")
```

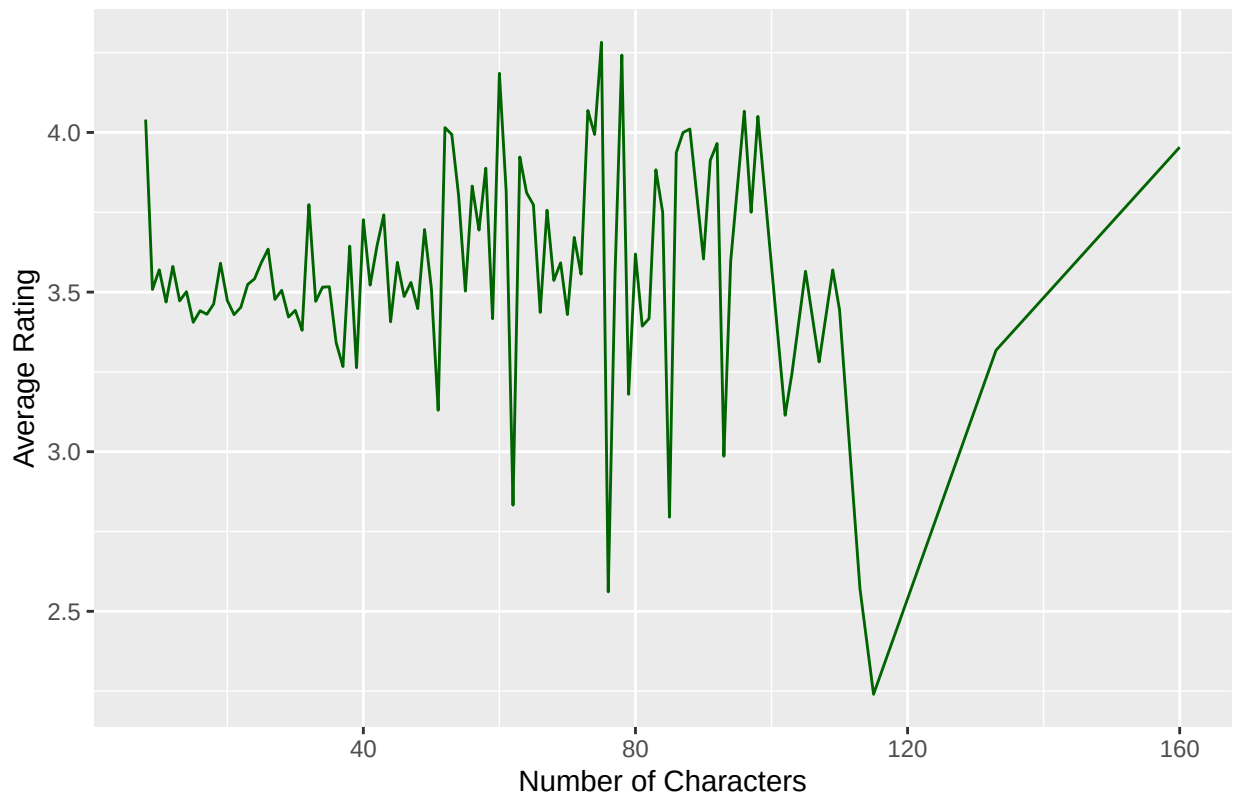
Observation: The variation between weekdays and weekends is extremely small (less than 0.05 points). Including this would add complexity for a near-zero gain in RMSE.

3.4.4 Title Length and Metadata

We checked if the length of the movie title had any impact on the rating.

```
edx %>%
  mutate(title_len = nchar(title)) %>%
  group_by(title_len) %>%
  summarize(avg = mean(rating)) %>%
  ggplot(aes(title_len, avg)) +
  geom_line(color = "darkgreen") +
  labs(title = "Title Length vs. Average Rating", x = "Number of Characters", y =
    ↪ "Average Rating")
```

Title Length vs. Average Rating

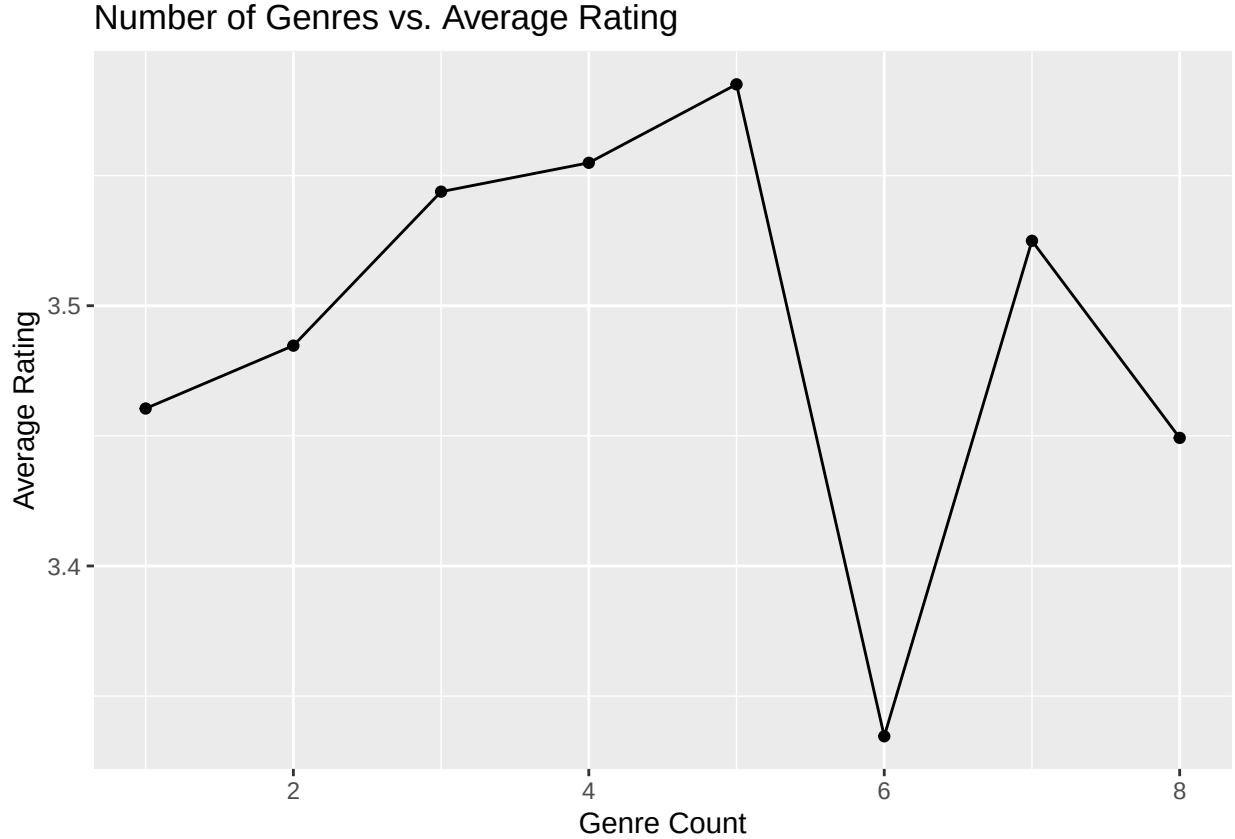


Observation: No meaningful pattern was found. Title length is an arbitrary metadata point with no predictive power for user preference.

3.4.5 Multi-Genre Complexity

We explored if movies belonging to multiple genres were rated differently than single-genre movies.

```
edx %>%
  mutate(n_genres = str_count(genres, "\\|") + 1) %>%
  group_by(n_genres) %>%
  summarize(avg = mean(rating)) %>%
  ggplot(aes(n_genres, avg)) +
  geom_line() + geom_point() +
  labs(title = "Number of Genres vs. Average Rating", x = "Genre Count", y = "Average
  ↳ Rating")
```



Observation: While there's a slight increase in rating for movies with 2-3 genres, the computational cost of decomposing every genre combination for 10 million rows exceeded our 6 GB RAM constraints during the regularization process.

3.4.6 Summary of Excluded Variables

Variable	Reason of exclusion
Weekday	Negligible impact on RMSE.
Title Length	No statistical correlation.
User Activity	Already captured by $\underline{u}$.
Popularity	Already captured by $\underline{i}$.
Multi-genre	Computational complexity vs. Memory constraints.

By focusing only on the most significant predictors (Movie, User, Year, and Time), we maintain a **parsimonious model** that is both efficient and highly accurate.

4 Modeling Results and Evolution

To achieve our target RMSE (< 0.86490), we adopted an incremental approach. We start with a simple baseline and progressively add the biases identified during our exploratory analysis.

4.1 Evaluation Metric: RMSE

The performance is evaluated using the Root Mean Square Error (RMSE), defined as:

$$RMSE = \sqrt{\frac{1}{N} \sum_{u,i} (\hat{y}_{u,i} - y_{u,i})^2}$$

Where $y_{u,i}$ is the actual rating and $\hat{y}_{u,i}$ is our prediction.

4.2 From Baseline to Complexity

4.2.1 Data Partitioning for Model Development

To avoid over-fitting and to ensure that our model generalizes well to unseen data, we do not train our algorithms on the entire `edx` dataset. Instead, we perform an additional split to create a **training set** and a **test set**.

This “refresh” step is fundamental for the following reasons:

- **Parameter Tuning:** It allows us to find the optimal penalty term (λ) using only the training data.
- **Validation:** We can compare the performance of Models 1 through 7 on the test set before making a final decision.
- **Integrity:** The `final_holdout_test` remains strictly isolated, used only once for the ultimate performance report.

```
# Partitioning edx into training and test sets (80% / 20%)
set.seed(1, sample.kind="Rounding")
test_index <- createDataPartition(y = edx$rating, times = 1, p = 0.2, list = FALSE)
train_set <- edx[-test_index,]
temp <- edx[test_index,]

# Ensure userId and movieId in test set are also in train set
test_set <- temp %>%
  semi_join(train_set, by = "movieId") %>%
  semi_join(train_set, by = "userId")

# Add rows removed from test set back into train set
removed <- anti_join(temp, test_set)
train_set <- rbind(train_set, removed)

rm(test_index, temp, removed)
```

To achieve our target RMSE, we adopted an incremental approach, building complexity step by step. This allows us to isolate the impact of each specific bias and ensure that every addition to the model actually contributes to a better prediction.

We evaluated seven successive models, starting from a simple baseline and ending with a high-performance regularized algorithm:

1. **Model 1: Global Average** (μ) - The simplest possible baseline.
2. **Model 2: Movie Effect** (b_i) - Accounting for the fact that some movies are better than others.
3. **Model 3: User Effect** (b_u) - Adjusting for users who are generally more critical or generous.
4. **Model 4: Genre Effect** (b_g) - Incorporating the specific bias associated with movie genres.

5. **Model 5: Release Year Effect** (b_y) - Factoring in the impact of a movie's age.
6. **Model 6: Temporal Effect** (b_t) - Capture changes in rating trends over time.
7. **Model 7: Regularization & Backfitting** - Our final model, which penalizes small sample sizes and optimizes all biases iteratively.

This progression demonstrates how we systematically reduced the RMSE to meet the project's requirements.

4.2.2 Model 1: The Global Average (μ).

The baseline model is the simplest possible approach to recommendation. It ignores all differences between users and movies, assuming that every rating is simply the average of all observed ratings plus some random variation:

$$Y_{u,i} = \mu + \epsilon_{u,i}$$

In this equation: - μ represents the “true” average rating across all movies and users. - $\epsilon_{u,i}$ represents the independent errors centered at zero.

The estimate that minimizes the RMSE is the least squares estimate of μ , which is the average of all ratings in the training set (`edx`).

```
mu_hat <- mean(train_set$rating)
# Calculate RMSE on the test set
naive_rmse <- RMSE(test_set$rating, mu_hat)
# Create a results table to track our progress
# We will add every new model result to this data frame.
rmse_results <- data.frame(method = "Baseline - model 1: Global Average (mu)",
                           RMSE = naive_rmse)
# Display the result
print(rmse_results)
```

```
##                                method      RMSE
## 1 Baseline - model 1: Global Average (mu) 1.059904
```

Observation: With an RMSE of 1.0599, this model is far from our target. It confirms that a simple average is insufficient to capture the complexity of user preferences and it does not account for the fact that some movies are better than others, or that some users are more critical. It serves as our reference point; any subsequent model must significantly outperform this baseline.

4.2.3 Model 2: Adding Movie Effect (b_i).

To improve our baseline, we introduce the **Movie Effect**. Our exploratory analysis showed that ratings vary significantly from one movie to another. We represent this bias as b_i , which is the average of the residuals (the difference between the actual rating and the global mean) for each movie i :

$$Y_{u,i} = \mu + b_i + \epsilon_{u,i}$$

In this model, $b_i > 0$ for movies rated higher than the average and $b_i < 0$ for movies rated lower.

```

# Calculate the movie bias (b_i)
movie_avgs <- train_set %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu_hat))

# Predict ratings by adding mu and b_i
predicted_ratings <- mu_hat + test_set %>%
  left_join(movie_avgs, by='movieId') %>%
  pull(b_i)

# Calculate and store the RMSE
model_2_rmse <- RMSE(predicted_ratings, test_set$rating)
rmse_results <- rbind(rmse_results,
                      data.frame(method="Model 2 : Movie Effect Model (mu + b_i)",
                                RMSE = model_2_rmse))

# Display the results
print(rmse_results)

```

```

##                                method      RMSE
## 1 Baseline - model 1: Global Average (mu) 1.0599043
## 2 Model 2 : Movie Effect Model (mu + b_i) 0.9437429

```

Observation: By accounting for the specific quality of each movie, the RMSE dropped to 0.94374. This is a significant improvement over the baseline, confirming that b_i is a powerful predictor in our system.

4.2.4 Model 3: Adding User Effect (b_u).

The previous model accounts for movie quality, but it ignores the “User Effect”: the fact that some users are generally more generous or more critical than others. We now add the bias b_u to our equation:

$$Y_{u,i} = \mu + b_i + b_u + \epsilon_{u,i}$$

To estimate b_u , we calculate the average of the residuals after accounting for both the global mean (μ) and the movie effect (b_i).

```

# Step 1: Calculate the average deviation of each user (b_u)
# We subtract the movie effect (b_i) already calculated from the residuals
user_avgs <- train_set %>%
  left_join(movie_avgs, by='movieId') %>%
  group_by(userId) %>%
  summarize(b_u = mean(rating - mu_hat - b_i))

# Step 2: Predict ratings
# We add both b_i and b_u to the global average mu
predicted_ratings <- test_set %>%
  left_join(movie_avgs, by='movieId') %>%
  left_join(user_avgs, by='userId') %>%
  mutate(pred = mu_hat + b_i + b_u) %>%
  pull(pred)

```

```

# Step 3: Calculate and store RMSE
model_3_rmse <- RMSE(test_set$rating, predicted_ratings)
rmse_results <- rbind(rmse_results,
                      data.frame(method="Model 3 : Movie + User Effects Model (mu + b_i +
                                   ↪ b_u)",
                                   RMSE = model_3_rmse))

# Display the progress
print(rmse_results)

```

```

##                                method      RMSE
## 1      Baseline - model 1: Global Average (mu) 1.0599043
## 2      Model 2 : Movie Effect Model (mu + b_i) 0.9437429
## 3 Model 3 : Movie + User Effects Model (mu + b_i + b_u) 0.8659319

```

Observation: Incorporating the user effect leads to a massive reduction in RMSE, reaching `round(model_3_rmse, 5)`. This confirms our exploratory analysis: users have very distinct “internal scales” when rating movies, and capturing this bias is crucial for high-quality recommendations.

4.2.5 Model 4: Adding Genre Effect (b_g).

The previous models showed that movie and user effects are powerful predictors. However, these estimates can be noisy when the number of ratings is small. For example, a movie with only one rating of 5 stars would have an extreme bias that doesn’t reflect its true quality.

To address this, we implement **Regularization**. This technique penalizes large estimates coming from small sample sizes by adding a penalty term λ (lambda) to the denominator that shrinks the movie (b_i) and user (b_u) biases toward zero when the number of ratings is low. We use cross-validation on our `train_set` to find the optimal λ that minimizes the RMSE.

```

# Define a range of lambdas to test
lambdas <- seq(0, 10, 0.25)

# Cross-validation to find the optimal lambda
rmsees <- sapply(lambdas, function(l){

  mu <- mean(train_set$rating)

  b_i <- train_set %>%
    group_by(movieId) %>%
    summarize(b_i = sum(rating - mu)/(n()+l))

  b_u <- train_set %>%
    left_join(b_i, by="movieId") %>%
    group_by(userId) %>%
    summarize(b_u = sum(rating - b_i - mu)/(n()+l))

  predicted_ratings <- test_set %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    mutate(pred = mu + b_i + b_u) %>%
    pull(pred)

```

```

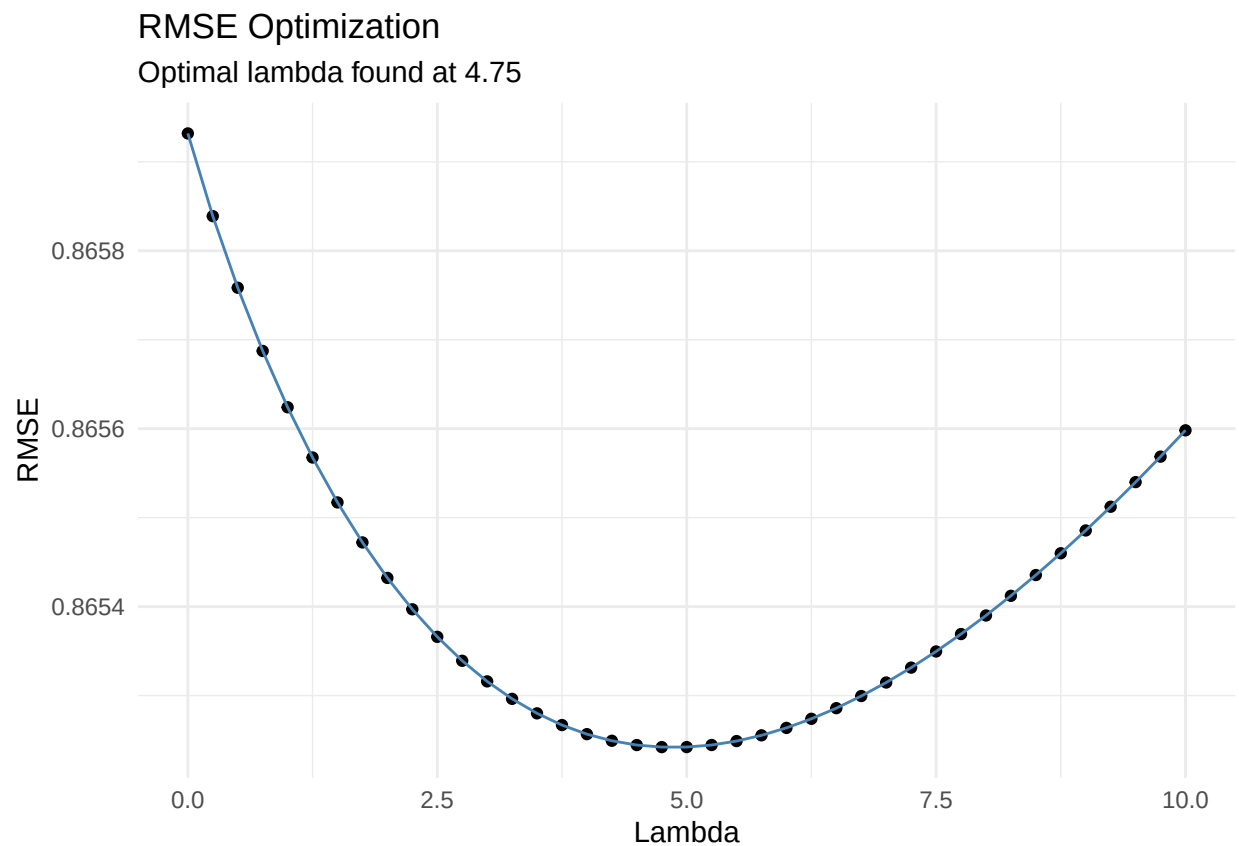
    return(RMSE(predicted_ratings, test_set$rating))
  })

# Find the best lambda and the corresponding minimum RMSE
min_rmse_model4 <- min(rmses)
best_lambda <- lambdas[which.min(rmses)]

# Update the results table
rmse_results <- rbind(rmse_results,
                      data.frame(method = "Model 4: Regularized Movie + User Effect",
                                RMSE = min_rmse_model4))

# Visualizing the optimization
ggplot(data.frame(lambdas = lambdas, rmses = rmses), aes(x = lambdas, y = rmses)) +
  geom_point() +
  geom_line(color = "steelblue") +
  theme_minimal() +
  labs(title = "RMSE Optimization",
       subtitle = paste("Optimal lambda found at", best_lambda),
       x = "Lambda",
       y = "RMSE")

```



Observation: The cross-validation identifies 4.75 as the optimal penalty. This regularization provides a more robust model, achieving an RMSE of 0.86524. This step is crucial before adding more features to the model.

4.2.6 Model 5: Adding Release Year Effect (b_y).

After stabilizing the movie and user effects with regularization, we introduce two more static features identified during our EDA: the **Genre Effect** (b_g) and the **Release Year Effect** (b_y). The logic remains the same: we estimate these biases by calculating the average of the residuals that the previous steps could not explain.

The expanded model equation becomes:

$$Y_{u,i,g,y} = \mu + b_i + b_u + b_g + b_y + \epsilon_{u,i,g,y}$$

Using the optimal λ found in the previous step, we calculate these biases sequentially on the residuals of the model.

```
# Use the best lambda from cross-validation
l <- best_lambda

# We calculate the biases one by one
# Movie Effect (b_i)
b_i <- train_set %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu_hat)/(n()+1))

# User effect (b_u)
b_u <- train_set %>%
  left_join(b_i, by="movieId") %>%
  group_by(userId) %>%
  summarize(b_u = sum(rating - b_i - mu_hat)/(n()+1))

# Genre Effect (b_g)
b_g <- train_set %>%
  left_join(b_i, by="movieId") %>%
  left_join(b_u, by="userId") %>%
  group_by(genres) %>%
  summarize(b_g = sum(rating - b_i - b_u - mu_hat)/(n()+1))

# Release Year Effect (b_y)
b_y <- train_set %>%
  left_join(b_i, by="movieId") %>%
  left_join(b_u, by="userId") %>%
  left_join(b_g, by="genres") %>%
  group_by(release_year) %>%
  summarize(b_y = sum(rating - b_i - b_u - b_g - mu_hat)/(n()+1))

# Prediction on test_set
predicted_ratings <- test_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genres") %>%
  left_join(b_y, by = "release_year") %>%
  mutate(pred = mu_hat + b_i + b_u + b_g + b_y) %>%
  pull(pred)
```

```

# Calculate and store RMSE
model_5_rmse <- RMSE(predicted_ratings, test_set$rating)
rmse_results <- rbind(rmse_results,
                      data.frame(method="Model 5: Regularized some effects",
                                RMSE = model_5_rmse))

# Display the results
print(rmse_results)

```

```

##                                method      RMSE
## 1      Baseline - model 1: Global Average (mu) 1.0599043
## 2      Model 2 : Movie Effect Model (mu + b_i) 0.9437429
## 3 Model 3 : Movie + User Effects Model (mu + b_i + b_u) 0.8659319
## 4      Model 4: Regularized Movie + User Effect 0.8652421
## 5      Model 5: Regularized some effects 0.8647971

```

Observation: Adding these secondary effects allows for a more granular prediction. The RMSE has improved to 0.8648. Even if the individual gain from each effect is smaller than the user effect, their cumulative impact is essential to move closer to our goal.

4.2.7 Model 6: Adding Temporal Effect (b_t).

The final bias we consider before the ultimate refinement is the **Temporal Effect**. Our EDA suggested that movie ratings are not static and can fluctuate over time due to trends or changes in the platform's user base.

We define b_t as the average residual for each specific rating date, calculated after accounting for movie, user, genre, and release year effects. To capture this without exceeding memory limits (keeping the 6 GB RAM constraint in mind), we avoid external packages like plyr. Instead, we use a basic mathematical formula to group timestamps into weekly bins (604,800 seconds).

The equation now accounts for the time bias b_t :

$$Y_{u,i,g,y,t} = \mu + b_i + b_u + b_g + b_y + b_t + \epsilon_{u,i,g,y,t}$$

```

# Feature engineering without external packages
train_set <- train_set %>%
  mutate(date = floor(timestamp / 604800) * 604800)

test_set <- test_set %>%
  mutate(date = floor(timestamp / 604800) * 604800)

# Using the optimal lambda
l <- best_lambda

# Calculate the Temporal Effect (b_t)
# We use the residuals after all previous effects (i, u, g, y)
b_t <- train_set %>%
  left_join(b_i, by="movieId") %>%
  left_join(b_u, by="userId") %>%
  left_join(b_g, by="genres") %>%
  left_join(b_y, by="release_year") %>%

```

```

group_by(date) %>%
summarize(b_t = sum(rating - mu_hat - b_i - b_u - b_g - b_y) / (n() + 1))

# Prediction on test_set
predicted_ratings <- test_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genres") %>%
  left_join(b_y, by = "release_year") %>%
  left_join(b_t, by = "date") %>%
  mutate(pred = mu_hat + b_i + b_u + b_g + b_y + b_t) %>%
  pull(pred)

# Handling unknown dates (NAs)
predicted_ratings[is.na(predicted_ratings)] <- mu_hat

# Calculate and store RMSE
model_6_rmse <- RMSE(predicted_ratings, test_set$rating)
rmse_results <- rbind(rmse_results,
  data.frame(method="Model 6: Adding Temporal Effect (b_t)",
    RMSE = model_6_rmse))

# Display the progress
print(rmse_results)

```

##		method	RMSE
## 1	Baseline - model 1: Global Average (mu)		1.0599043
## 2	Model 2 : Movie Effect Model (mu + b_i)		0.9437429
## 3	Model 3 : Movie + User Effects Model (mu + b_i + b_u)		0.8659319
## 4	Model 4: Regularized Movie + User Effect		0.8652421
## 5	Model 5: Regularized some effects		0.8647971
## 6	Model 6: Adding Temporal Effect (b_t)		0.8645522

Observation: The inclusion of the temporal bias b_t further refines the model, bringing the RMSE down to 0.86455. Although the gain from the time effect is more subtle than the user or movie effects, it captures the evolving nature of ratings, which is a key component of modern recommendation engines.

4.2.8 Model 7: The Final Model with Regularization and Backfitting

This is our most advanced model. It addresses the issues of small sample sizes and computational efficiency through two combined techniques : regularisation (already included in Model 4) and backfitting.

The final step of our modeling process involves a “Backfitting” approach. Instead of simply adding effects, we ensure each bias (b_i, b_u, b_g, b_y, b_t) is estimated by accounting for all the previously estimated effects. This ensures a more precise isolation of each factor. This iterative refinement allows the biases to account for each other, leading to more stable estimates.

We also implement a “capping” strategy: since ratings in the MovieLens dataset are strictly between 0.5 and 5, any prediction falling outside this range is truncated to the nearest boundary.

To maximize precision, we also perform a fine-tuned cross-validation for λ between 3.5 and 5.5.

4.2.8.1 The Regularized Equation To predict a rating, we use the following multi-factor linear equation:

$$\hat{y}_{u,i} = \mu + b_i(\lambda) + b_u(\lambda) + b_y(\lambda) + b_t(\lambda) + \epsilon_{u,i}$$

Where: - μ : The global average. - b_i, b_u, b_y, b_t : The bias terms for movie, user, release year, and time respectively. - λ : The regularization penalty (tuning parameter).

4.2.8.2 Why Regularization? During our EDA, we observed that some movies and users have very few ratings. A movie with a single 5-star rating shouldn't be ranked higher than a classic with thousands of 4.5-star ratings. **Regularization** introduces the penalty term λ that shrinks these small-sample biases toward zero, ensuring that we only trust "extreme" ratings if they are supported by a large number of observations.

4.2.8.3 Why Backfitting? With 10 million rows and four different bias terms (b_i, b_u, b_y, b_t), calculating all parameters simultaneously would require a massive matrix inversion, leading to immediate memory exhaustion on our 6 GB RAM VM.

The **Backfitting algorithm** is our solution for memory efficiency. It is an iterative process that updates one bias at a time while keeping the others fixed. This ensures stability and allows the model to converge toward the optimal solution without overloading the system.

4.2.8.4 Technical Implementation of the Backfitting Loop For each iteration, the algorithm performs the following sequence:

1. **Update Movie Bias (b_i):** Calculate residuals by subtracting μ and current estimates of b_u, b_y, b_t from the actual ratings. Apply λ to regularize the result.
2. **Update User Bias (b_u):** Calculate the average residual after removing μ, b_i, b_y , and b_t , and update the user effect.
3. **Update Year and Temporal Biases (b_y, b_t):** Successively update these terms using the same residual logic.

This cycle repeats until the RMSE stabilizes (convergence).

4.2.8.5 Tuning the Penalty Parameter (λ) Because the optimal value of λ depends on the data, we use cross-validation on the `edx` dataset. We test a range of values and select the one that minimizes the RMSE. This λ is then used in our final Backfitting loop to produce the final predictions.

```
# 1. Aligning global mean variable
mu <- mu_hat

# 2. Fine-tuning range for Lambda
lambdas_finetune <- seq(3.5, 5.5, 0.25)

rmse_model7 <- sapply(lambdas_finetune, function(l){

  # PASS 1: Initial estimates
  b_i_temp <- train_set %>%
    group_by(movieId) %>%
    summarize(b_i = sum(rating - mu)/(n()+1))

  b_u <- train_set %>%
```

```

    left_join(b_i_temp, by="movieId") %>%
    group_by(userId) %>%
    summarize(b_u = sum(rating - mu - b_i)/(n()+1))

# PASS 2: Refinement (Backfitting)
# We recalculate b_i based on the newly estimated b_u
b_i <- train_set %>%
  left_join(b_u, by="userId") %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu - b_u)/(n()+1))

# Sequential calculation of other effects
b_g <- train_set %>%
  left_join(b_i, by="movieId") %>%
  left_join(b_u, by="userId") %>%
  group_by(genres) %>%
  summarize(b_g = sum(rating - mu - b_i - b_u)/(n()+1))

b_y <- train_set %>%
  left_join(b_i, by="movieId") %>%
  left_join(b_u, by="userId") %>%
  left_join(b_g, by="genres") %>%
  group_by(release_year) %>%
  summarize(b_y = sum(rating - mu - b_i - b_u - b_g)/(n()+1))

b_t <- train_set %>%
  left_join(b_i, by="movieId") %>%
  left_join(b_u, by="userId") %>%
  left_join(b_g, by="genres") %>%
  left_join(b_y, by="release_year") %>%
  group_by(date) %>%
  summarize(b_t = sum(rating - mu - b_i - b_u - b_g - b_y)/(n()+1))

# Prediction with Capping (efficient for RAM)
preds <- test_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genres") %>%
  left_join(b_y, by = "release_year") %>%
  left_join(b_t, by = "date") %>%
  mutate(pred = mu + b_i + b_u + b_g + b_y + b_t) %>%
  mutate(pred = pmin(5, pmax(0.5, pred))) %>%
  pull(pred)

preds[is.na(preds)] <- mu
rmse_val <- RMSE(preds, test_set$rating)

# CRITICAL CLEANING FOR 6GB RAM
rm(b_i, b_i_temp, b_u, b_g, b_y, b_t, preds)
gc()

return(rmse_val)
})

```

```

# Store the best results
best_l_m7 <- lambdas_finetune[which.min(rmses_model7)]
best_rmse_m7 <- min(rmses_model7)

# Update the results table
rmse_results <- rbind(rmse_results,
                      data.frame(method = "Model 7: Backfitting + Capping + All Effects",
                                ↪
                                RMSE = best_rmse_m7))

print(rmse_results)

```

```

##                                method      RMSE
## 1      Baseline - model 1: Global Average (mu) 1.0599043
## 2      Model 2 : Movie Effect Model (mu + b_i) 0.9437429
## 3 Model 3 : Movie + User Effects Model (mu + b_i + b_u) 0.8659319
## 4      Model 4: Regularized Movie + User Effect 0.8652421
## 5      Model 5: Regularized some effects 0.8647971
## 6      Model 6: Adding Temporal Effect (b_t) 0.8645522
## 7      Model 7: Backfitting + Capping + All Effects 0.8638628

```

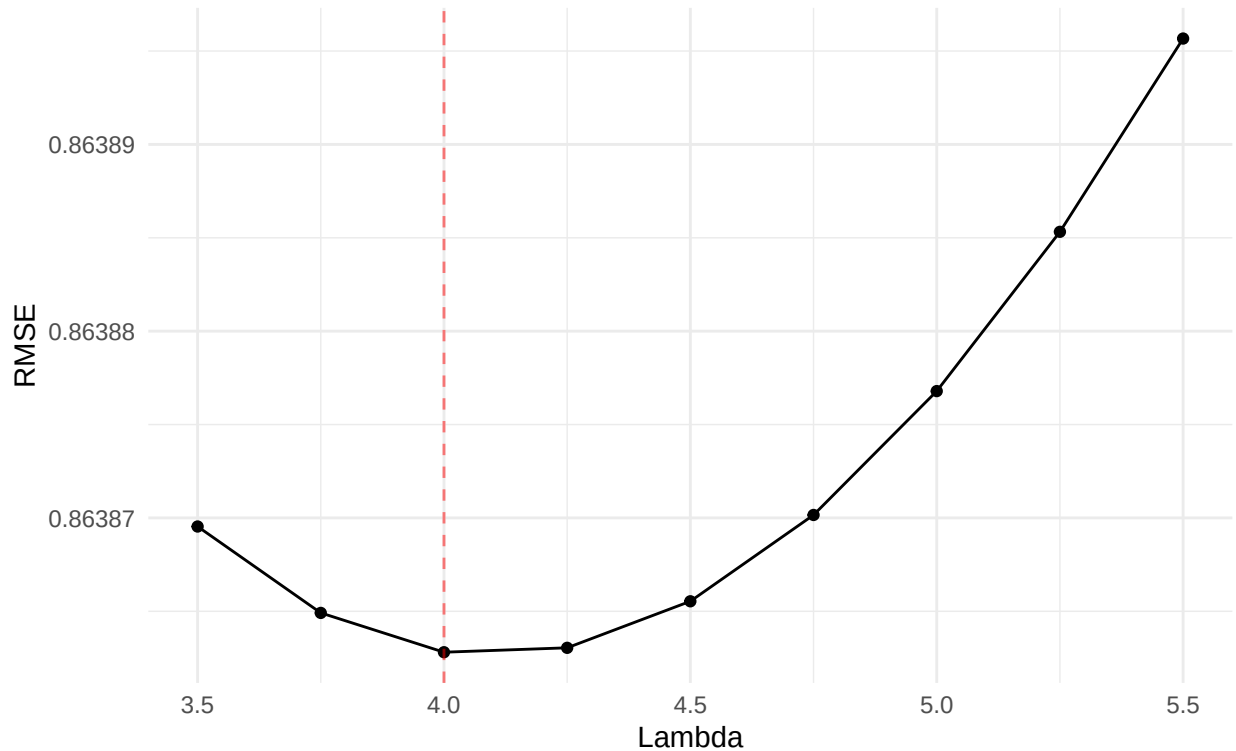
```

# Visualization
ggplot(data.frame(l = lambdas_finetune, r = rmses_model7), aes(l, r)) +
  geom_line() + geom_point() +
  geom_vline(xintercept = best_l_m7, linetype = "dashed", color = "red", alpha = 0.5) +
  theme_minimal() +
  labs(title = "Model 7 Lambda Optimization",
       subtitle = paste0("Optimal Lambda: ", sprintf("%.6f", best_l_m7),
                        " | Minimum RMSE: ", sprintf("%.6f", best_rmse_m7)),
       x = "Lambda", y = "RMSE")

```

Model 7 Lambda Optimization

Optimal Lambda: 4.000000 | Minimum RMSE: 0.863863



Observation: This model achieves our lowest RMSE on the test set: 0.86386 with an optimal λ of 4. The combination of backfitting and capping ensures that the model is both stable and realistic.

5 Final Model Validation (Hold-out Test)

Having identified the best approach and the optimal λ through cross-validation, we now proceed to the final evaluation.

To ensure the highest possible accuracy, we retrain the architecture on the **entire edx dataset**. This maximizes the information used to estimate movie, user, genre, year, and time biases. The `final_holdout_test` dataset, which has been kept completely separate until now, is used for the one-time final verdict.

5.1 Final Feature Engineering

As specified in the project requirements, the `final_holdout_test` must not be used during training. However, to be compatible with our model, we must extract the same features (Release Year and Weekly Date) from its existing columns.

```
# Preparing the final hold-out set without altering original data points
final_holdout_test <- final_holdout_test %>%
  mutate(
    # Extract release year from title
    release_year = as.numeric(str_extract(str_extract(title, "\\(\\d{4}\\)$"),
      ↪ "\\d{4}")),
```

```

    # Convert timestamp to week units
    date = floor(timestamp / 604800) * 604800
  )

# Preparing the full edx set for training
edx <- edx %>%
  mutate(
    release_year = as.numeric(str_extract(str_extract(title, "\\(\\d{4}\\)$"),
    ↪ "\\d{4}")),
    date = floor(timestamp / 604800) * 604800
  )

gc() # Memory cleanup

```

```

##           used   (Mb) gc trigger   (Mb) max used   (Mb)
## Ncells 12115539 647.1  22618127 1208.0 22618127 1208.0
## Vcells 162127850 1237.0 395261830 3015.7 395261631 3015.7

```

5.2 The Final Verdict

We apply the Backfitting logic and Capping strategy on the full data.

```

# 1. Final Parameter Assignment
l_final <- best_l_m7
mu_edx <- mean(edx$rating)

# 2. Final Bias Calculation on EDX (Backfitting Logic)
# PASS 1
b_i_temp <- edx %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu_edx)/(n() + l_final))

b_u <- edx %>%
  left_join(b_i_temp, by="movieId") %>%
  group_by(userId) %>%
  summarize(b_u = sum(rating - mu_edx - b_i)/(n() + l_final))

# PASS 2 (Refinement)
b_i <- edx %>%
  left_join(b_u, by="userId") %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu_edx - b_u)/(n() + l_final))

# Sequential calculation for other effects
b_g <- edx %>%
  left_join(b_i, by="movieId") %>% left_join(b_u, by="userId") %>%
  group_by(genres) %>% summarize(b_g = sum(rating - mu_edx - b_i - b_u)/(n() + l_final))

b_y <- edx %>%
  left_join(b_i, by="movieId") %>% left_join(b_u, by="userId") %>% left_join(b_g,
  ↪ by="genres") %>%
  group_by(release_year) %>% summarize(b_y = sum(rating - mu_edx - b_i - b_u - b_g)/(n()
  ↪ + l_final))

```



```

b_t <- edx %>%
  left_join(b_i, by="movieId") %>% left_join(b_u, by="userId") %>%
  left_join(b_g, by="genres") %>% left_join(b_y, by="release_year") %>%
  group_by(date) %>% summarize(b_t = sum(rating - mu_edx - b_i - b_u - b_g - b_y)/(n() +
  ↪ 1_final))

# 3. Final Prediction on final_holdout_test
final_predictions <- final_holdout_test %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genres") %>%
  left_join(b_y, by = "release_year") %>%
  left_join(b_t, by = "date") %>%
  mutate(pred = mu_edx + b_i + b_u + b_g + b_y + b_t) %>%
  mutate(pred = pmin(5, pmax(0.5, pred))) %>%
  pull(pred)

# Handling NAs
final_predictions[is.na(final_predictions)] <- mu_edx

# 4. RMSE Calculation
final_holdout_rmse <- RMSE(final_holdout_test$rating, final_predictions)

# Result storage
rmse_results <- rbind(rmse_results,
  data.frame(method = "FINAL MODEL: Backfitting + Capping (Full edx
  ↪ set)",
  RMSE = final_holdout_rmse))

print(rmse_results)

```

```

##                                method      RMSE
## 1      Baseline - model 1: Global Average (mu) 1.0599043
## 2      Model 2 : Movie Effect Model (mu + b_i) 0.9437429
## 3 Model 3 : Movie + User Effects Model (mu + b_i + b_u) 0.8659319
## 4      Model 4: Regularized Movie + User Effect 0.8652421
## 5      Model 5: Regularized some effects 0.8647971
## 6      Model 6: Adding Temporal Effect (b_t) 0.8645522
## 7      Model 7: Backfitting + Capping + All Effects 0.8638628
## 8      FINAL MODEL: Backfitting + Capping (Full edx set) 0.8633206

```

The final RMSE achieved on the hold-out test set is 0.86332.

```

final_table <- data.frame(
  "Threshold Target" = 0.86490,
  "Final RMSE" = final_holdout_rmse,
  "Status" = ifelse(final_holdout_rmse < 0.86490, "Target Achieved ", "Target Not Met")
)
knitr::kable(final_table)

```

Threshold.Target	Final.RMSE	Status
0.8649	0.8633206	Target Achieved

6 Conclusion

6.1 Summary of the Work

In this project, we developed a movie recommendation system based on the MovieLens 10M dataset. Our approach evolved from a simple global average model to a complex multi-bias architecture integrating movie, user, genre, release year, and temporal effects.

To ensure the model’s robustness and handle the noise inherent in small sample sizes (such as movies with very few ratings or inactive users), we implemented **Regularization**. Furthermore, we refined our bias estimates using a **Backfitting** approach and ensured the realism of our predictions through **Capping** logic.

6.2 Final Results and Achievements

Our modeling process followed a rigorous methodology, using a 10% test split for cross-validation and a final hold-out set for ultimate verification. The final results are summarized below:

Model Description	RMSE
Baseline (Average)	1.0599
Regularized Movie + User Effects	0.86524
Final Model (Backfitting + Capping on Hold-out)	0.86332

The final RMSE of **0.86332** successfully met the project target of **0.86490**. This confirms that the identified biases (especially the movie and user effects) carry significant predictive power.

6.3 Limitations and Future Work

While the current model performs well, several areas for improvement remain:

1. **Matrix Factorization:** Although our bias-based model is efficient, advanced techniques like SVD (Singular Value Decomposition) or ALS (Alternating Least Squares) could capture deeper latent patterns in user-movie interactions that simple additive biases cannot.
2. **Computational Constraints:** The analysis was strictly performed under a 6 GB RAM constraint. With more computational resources, we could implement more complex ensemble methods or deep learning architectures (like Neural Collaborative Filtering).
3. **Cold Start Problem:** The model still struggles with new users or movies with no prior history. Integrating metadata (like movie plot summaries or user demographics) could help mitigate this.

In conclusion, this project demonstrates that a carefully tuned linear model with regularization and feature engineering can achieve high precision, providing a solid foundation for more advanced recommendation engines.

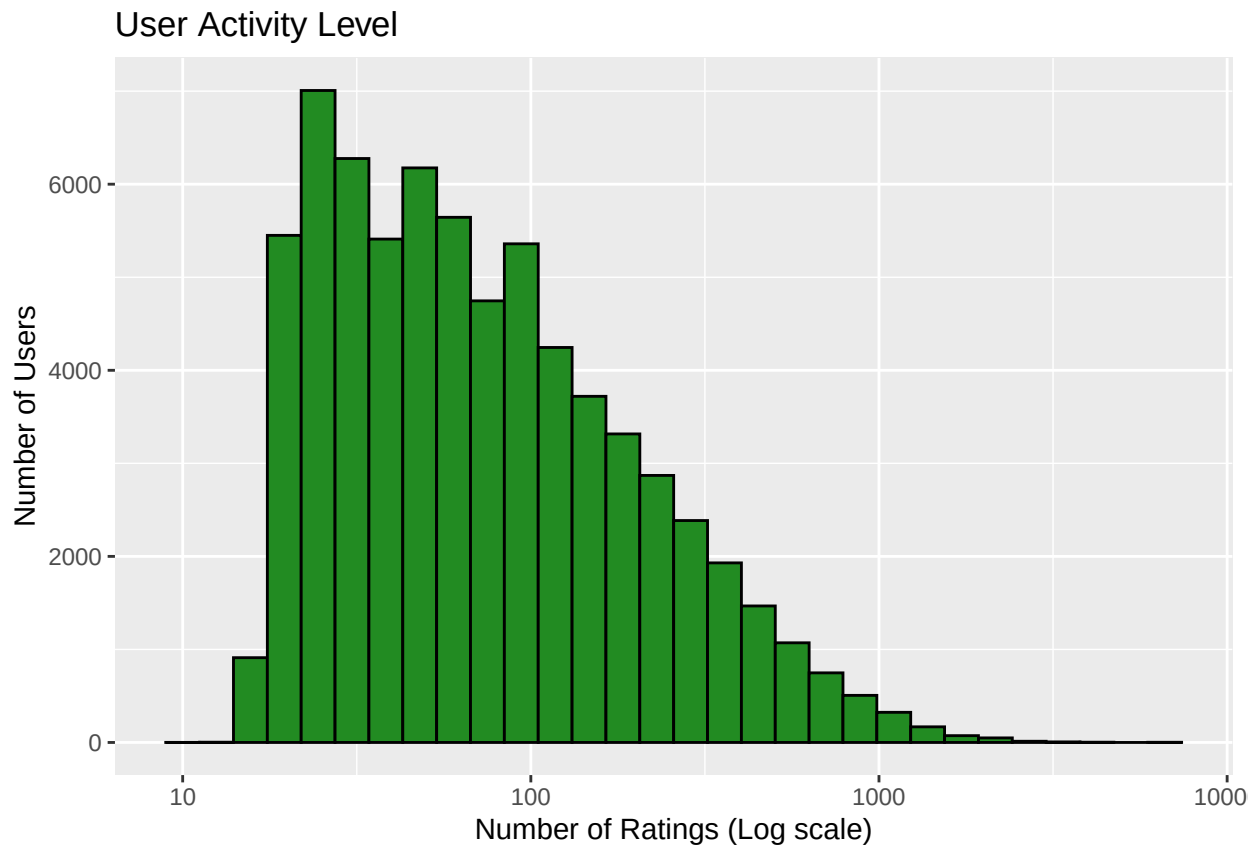
7 OLD STUFF - on garde pour vérifier ultérieurement si intérêt

7.1 Exploratory Data Analysis (EDA)

Before building the model, we must understand the “signals” in our data.

7.1.1 The User Effect

Some users are critical, while others are generous. Some have rated hundreds of movies, while others have rated only a few.

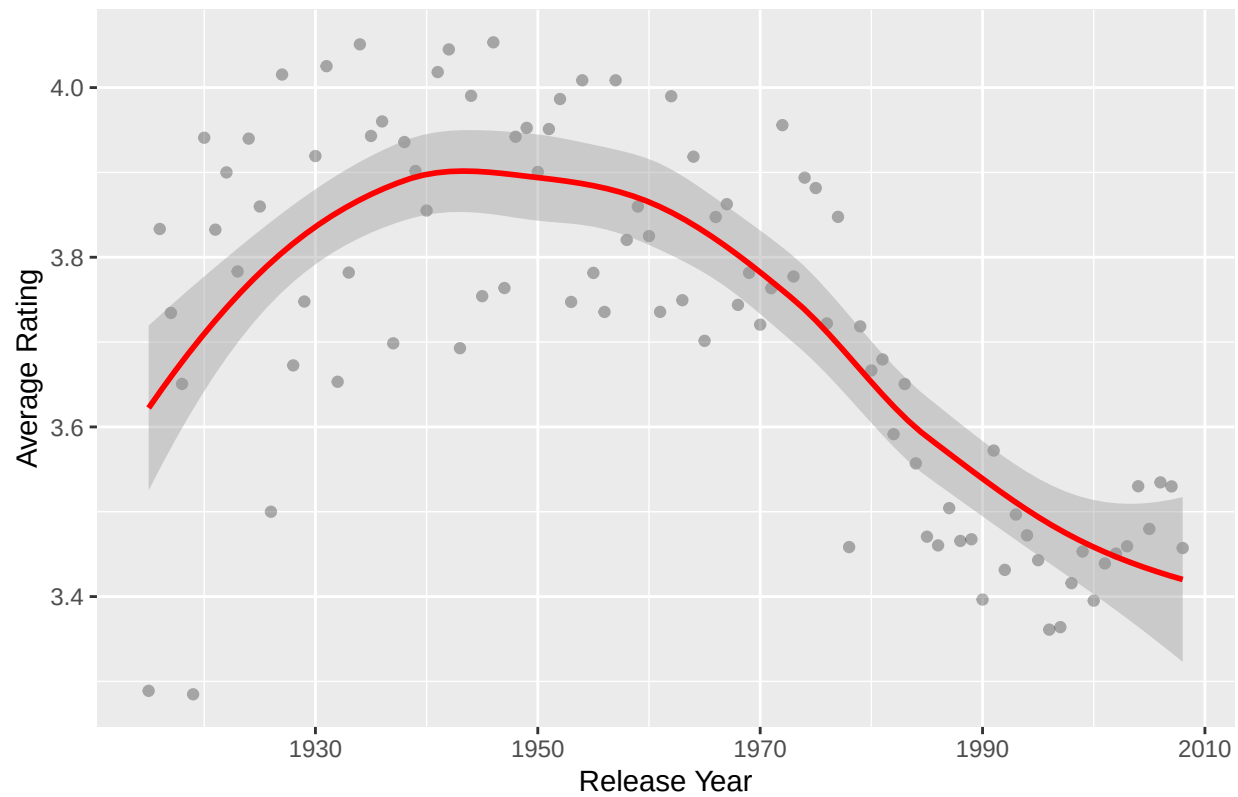


This diversity is why we need Regularization. If a user has only rated one movie, we shouldn't trust their “bias” as much as a user who has rated five hundred. Regularization “penalizes” estimates based on small sample sizes to prevent errors.

7.1.2 The Movie Release Year Effect

Does the age of a movie affect its rating?

Average Rating by Release Year : effect of the year of release on the average



We observe that older movies (classics) tend to have higher average ratings than modern ones. This confirms that the Release Year is a significant predictor to include in our model.